

eosc-basic-compliance

Test suite, workflow and configuration overview

Clean clone, 26 September 2026 — documentation commit [a0be8d9](#), 339 test cases

Repository: github.com/marioarele/eosc-basic-compliance

Contents

1. Language and framework · 2. What is actually tested · 3. The ten checklist points · 4. Link-following depth · 5. Configuration and customisation · 6. Continuous integration, and running it from the GitHub web interface · 7. Tests that exist because of real defects · 8. Running it · 9. Setting it up on macOS · 10. The published figures · 11. Worked examples · Sources

What the test suite covers, how the tool is configured, and what the published figures actually say. Every figure was measured on 25 September 2026 against a clean clone of the repository state this document is committed with (from commit [ada1b4a](#)). The test count was re-checked on 26 September 2026 against a clean clone of commit [0cc33c2](#), which added the command-line help tests: **339 test cases, all passing in CI**. No node website was contacted to prepare this edition.

👉 **Installation, configuration and run guide** — how to install and run the tool. Section 7 of that guide is the short version of this document.

👉 **Analysis workflow** — what the tool does once running: the collection sequence, the evidence model, and the exact branch conditions behind each of the ten checklist points.

In one line. A Python tool that checks EOSC Node Landing Pages against *Node Landing Page Verification Checklist v3.0*, with a pytest suite of 339 hermetic test cases that never touch a node's website, and at most two bounded hops of link following.

On "the previous edition". This document has been revised several times while the tool was being built. The text of this edition is also committed to the repository, as [docs/TEST-SUITE.md](#). Where the text below says a previous edition was wrong, that correction is kept rather than silently dropped, because a figure that was quoted in a meeting is worth being able to trace.

What changed since the 21 September edition. The suite grew from 111 to **339 cases** and gained three files: [tests/test_names.py](#) (64 cases, the approved-name matcher), [tests/test_nodes.py](#) (13 cases, guarding the node configuration) and [tests/test_privacy.py](#) (51 cases, personal-data masking). Four things the previous edition described as true are no longer true, and each is corrected in place below rather than quietly dropped:

Previous edition said	Now
<code>--only</code> rewrites the shared report — a known defect, unfixed	Fixed , and pinned by four tests (section 5)
Two latent defects in report.py , known and unfixed	Fixed at commit 014682c (section 7)
<code>uv.lock</code> is not committed, so runs are not reproducible	Committed ; CI installs with <code>--locked</code> (section 9)
The web form cannot reach depth 2	It can — the <code>depth</code> input now offers 2 (section 6)

Since the 24 September edition. Personal data is now masked in the evidence and in every report (section 5), and the published run was masked at commit [ada1b4a](#) with every verdict unchanged (section 10). BBMRI-ERIC's configured URL changed to a [dev3](#). address on 25 September, and the published run was collected from the old one, so rebuilding it needs the old node list (section 5). The suite grew from 326 to 329 cases for these changes, to 333 with four tests that keep the command-line help complete, and to 339 with the tests for the URL-mismatch warning and for switching the default checklist revision (section 11).

The node list also grew from nine to **thirteen** (CERN and EOSC Node Czechia on 21 September, Italy and Slovakia on 24 September), and the published report in [results/](#) now covers twelve of them. Italy was skipped because its domain did not resolve on 24 September; section 10 has the details. The suite's runtime rose from ~0.2 s to ~5.9 s for a reason worth knowing (section 2). Every figure below was re-measured rather than carried over, and section 7 records a further defect found while preparing this edition, since fixed.

1. Language and framework

Python, requiring 3.12 or newer, developed on 3.14. Packaged with hatchling and managed with [uv](#). Roughly **4,498 lines across seven modules** ([checks.py](#) 1142, [report.py](#) 1019, [cli.py](#) 819, [fetch.py](#) 712, [privacy.py](#) 383, [names.py](#) 247, [patterns.py](#) 176), plus **3,460 lines of tests**. Tests are now about three quarters the size of the code they exercise.

The suite is **pytest** — the PyUnit lineage rather than JUnit, but it does not use [unittest.TestCase](#) classes at all. Tests are plain functions with bare [assert](#) statements. There is no [setUp/tearDown](#); shared setup is a few small helper functions that build page evidence.

JUnit appears only as an output format. Continuous integration runs [pytest -q --junitxml=results/junit.xml](#), and the [mikepenz/action-junit-report](#) action turns that XML into the GitHub Checks summary. So you get JUnit-style reporting in the Actions tab without JUnit being the test framework. [ruff](#) runs as a lint gate before pytest, and the XML is also kept as a build artifact.

Component	Choice
Language	Python ≥ 3.12, developed on 3.14 (declared in pyproject.toml)
Test framework	pytest 8 (plain functions, no TestCase classes)
Report format	JUnit XML, published to GitHub Checks
Lint gate	ruff (E, F, I, B, SIM, UP), line length 100
Page rendering	Playwright + Chromium (headless)
HTML parsing	selectolax
Language detection	lingua-language-detector
CLI	typer
Config format	YAML (pyyaml)

2. What is actually tested

277 test functions, expanding to 339 executed cases (thirteen tests are parametrized). **None of them touch the network, and none of them open a browser.** This is the central design decision: most tests construct `PageEvidence` objects directly in memory — a synthetic page carrying the links, HTTP status and text a scenario needs. Nothing in the suite requests a real website, so the suite costs nothing and cannot fail because a node is down or because someone edited a page.

The absence of a browser requirement is verified rather than assumed: running the suite with `PLAYWRIGHT_BROWSERS_PATH` pointed at an empty directory still yields 339 passed, while `collect` fails with Playwright's "Executable doesn't exist" error. That is why the CI job installs no browser.

File	Cases	Covers
<code>test_names.py</code>	64	The approved-name matcher: separator tolerance, scoping to a node, anchoring, the committed default list and its provenance
<code>test_checks.py</code>	75	The checklist rules per point — 1, 3, 4, 5b, 5c, 6, 7 — plus cross-point invariants, that a point 3 summary never contradicts its own evidence, and the point 6 and 5b/5c cases found by the 24 September review
<code>test_report.py</code>	36	Matrix rendering, the dual-depth tables, Markdown escaping, table-breaking input
<code>test_crawl.py</code>	35	Link selection, host containment, whether following a link changes a verdict, the depth-2 budget, the depth-1 view
<code>test_cli.py</code>	50	Argument handling, node-id derivation, output isolation, report scope, <code>--skip</code> , command wiring, and that every option has help text, shared options are described alike, and <code>--help</code> lists the node ids; that evidence from a URL other than the configured one is flagged in every report
<code>test_checklist.py</code>	15	Provenance of the checklist: source document hash for every committed revision, point-to-rule mapping, version/filename convention, and that the default revision is one line that <code>--help</code> follows

File	Cases	Covers
test_nodes.py	13	The node configuration itself: required fields, unique ids, well-formed URLs, no two nodes sharing an eoscc.eu entry, every node's name covered by the scoped list
test_privacy.py	51	Personal-data masking: what is masked, what is kept, that evidence files and every report format are written masked, and that the committed evidence stays masked
Total	339	

[test_names.py](#) is the largest file in the suite, and deliberately so: point 3 is the only check that compares page text against an externally supplied list of official names, which makes it the check most able to produce a confident wrong answer. [test_nodes.py](#) is new since the node list grew — adding a node is now a configuration edit that the suite validates rather than a change nothing checks.

Why the suite takes ~5–6 seconds rather than ~0.2

The previous edition reported ~0.2 s. It is now **5–6 s** (5.2–5.9 s across four consecutive runs on 25 September 2026; 5.64–5.96 s across three on 24 September), and the cause is not a slow test but a change in what some tests do. A dozen or so tests in [test_names.py](#) and [test_cli.py](#) copy the **committed evidence** into a temporary directory and run a full [assess](#) through it, then read the resulting [results.json](#):

Slowest cases	Time
test_the_separator_rule_in_force_is_recorded	0.62 s
test_only_does_not_shrink_the_report_in_the_published_directory	0.35 s
test_with_no_flag_the_committed_default_is_used	0.35 s
test_a_full_run_records_no_selection	0.33 s

That is end-to-end assessment against real captured pages, which is why it costs real time — and it is still offline: disk and CPU only, no network and no browser. The trade is deliberate. A matcher tested only against synthetic strings would not have caught a report-scoping defect, and these are the tests that pin the [–only](#) fix.

The trade-off in the other direction remains explicit: the suite verifies that *the checklist rules are correctly applied to a page*, never that fetching works. Playwright's own behaviour is not covered, and that gap has cost something — see section 7.

Testing criteria

Six principles run through the suite.

- **Per-point rules with their near-misses.** For each decidable point, a passing case and the ways it can nearly pass. Point 4 has **six** tests because it is the sharpest point in the checklist: a correct node entry

passes, the federation index fails, the eossc.eu homepage fails, no link at all fails, a lookalike domain fails, and a domain merely *ending* in eossc.eu fails.

- **Nothing may silently pass.** One test asserts that points the checklist leaves to human judgement can never return PASS. Another asserts exactly one result per checklist point per node, so a point cannot quietly vanish from the matrix. A third blocks a PASS on point 3 when no approved-names list is in force — which now requires `--no-approved-names`, since a committed default list is used otherwise.
- **Absence is not concluded from a broken collection.** If a page did not render — a cookie overlay, a JavaScript shell — then a missing link cannot be reported as absent, because that describes the tool rather than the node. A companion test stops that safeguard becoming a blanket excuse for pages that did render.
- **Link following must not make PASS cheaper.** A whole block of tests asserts that following a link can only make a verdict better-evidenced, never more generous: a broken policy link fails, a navigation stub is not accepted as a policy document, and an About page cannot flip point 2, which asks the landing page itself to state those things.
- **The paperwork cannot drift from the code.** Twelve tests assert that the checklist YAML and the rules stay in step: the source document is committed and still hashes to what was transcribed, every point names the function that decides it, every function is claimed by exactly one point, and a revision must arrive as a new file. See section 5.
- **Regression pins for real defects.** At least ten tests exist because the tool actually got something wrong. Each carries the reason in its docstring rather than only an assertion. Five are described in section 7.
- **The configuration is part of the software.** Since the node list grew beyond the original nine, thirteen tests treat `nodes.yaml` as something that can be broken: a missing field, a duplicate id, a malformed URL, two nodes pointing at the same eossc.eu entry, or a node whose official name nothing in the scoped list covers all fail the suite rather than surfacing as a strange verdict later.

A guard that cannot fail is worse than no guard. Both dual-depth guards in `test_report.py` were confirmed by deliberate mutation — rendering the deep results into the shallow table, and forcing a stray heading back — and each failed exactly one test. A test written against the wrong part of the output passes for the wrong reason: one of these initially asserted against the tally line rather than the table row, and would have stayed green through the bug it was written to catch.

3. The ten checklist points

Checklist v3.0, dated 15 September 2026. The tool scores ten columns, because point 1 contains two separate requirements and is split into 1 and 1R.

Point	Requirement	Can a tool decide it?	Implemented by
1	Landing page publicly accessible, or reachable via EOSC AAI login	Yes, by inspection	check_1
1R	Resources pointed to by the page are also public or behind EOSC AAI	No — human judgement	check_1R
2	Scope, intended users and responsible organization are stated	No — human judgement	check_2

Point	Requirement	Can a tool decide it?	Implemented by
3	EOSC logo and official Tripartite-approved node name are visible	Partly — the name against a list, never the "clearly visible" judgement	check_3
4	Links to the node's own dedicated page on eosc.eu	Yes, by inspection	check_4
5a	Purpose description accessible in English for research resources	No — human judgement	check_5a
5b	Acceptable Use Policy (AUP) accessible	Partly	check_5b
5c	User Access Policy (UAP) accessible	Partly	check_5c
6	Means of contacting the node helpdesk	Partly	check_6
7	The page is in English	Yes, by inspection	check_7

The last column is not decoration. Each point declares its implementing function in the checklist YAML as [implemented_by](#), and a test asserts the mapping is complete in both directions, so a rule cannot exist without being explained and a point cannot be described without being applied. Run [basic-check points](#) to print this mapping from the file itself.

On the honesty of the output. Three of the ten points cannot be settled by a script at all and four only partly. Those are reported as [MANUAL_REVIEW](#) with the evidence attached rather than guessed at. Roughly 60% of cells land in review by design. A confident wrong verdict about a named organisation costs more to retract than an honest "a human must look".

4. Link-following depth

Two hops maximum, and the default is one. [--depth](#) is validated by the command line as an integer in the range 0 to 2, so an out-of-range value is refused rather than silently clamped.

A link is followed only when its target could settle a specific checklist point: an Acceptable Use Policy (5b), a User Access Policy (5c), a contact page (6), or an about page (2). Everything else is skipped, and every skip is recorded in the evidence so that a later "not found" is accountable rather than merely asserted.

--depth	What is fetched	Requests observed, the 9 published nodes, 21 Sep 2026
0	The landing page only	9
1 (default)	Plus links from it that could settle a point	31 (9 + 22)
2	Plus one further hop — a policy <i>index</i> that links on to the actual policy	45 (9 + 22 + 14)

Limit	Value	Purpose
Maximum depth	2	Enforced by the CLI, not by convention
Pages per node at depth 1	8	Configurable via <code>--max-children</code>
Pages per purpose at depth 1	2	<code>MAX_PER_PURPOSE</code> — two candidate AUP links is enough to decide
Pages per purpose at depth 2	1	<code>MAX_PER_PURPOSE_D2</code>
Run-wide second-hop budget	60	Configurable via <code>--fetch-budget</code> . A whole-run ceiling, not per node
Delay between children	1.2 s	<code>CHILD_DELAY_S</code>
Delay between grandchildren	1.5 s	<code>GRANDCHILD_DELAY_S</code>
Delay between nodes	2.0 s	Configurable via <code>--delay</code>
<code>robots.txt</code>	Honoured	Checked per host, since children may sit on other hosts
Excluded targets	—	Third-party hosts, PDFs and other binaries, the landing page itself
Screenshots of children	None	Keeps the extra cost to one request each

The payoff is not breadth but verification: a link labelled "Acceptable Use Policy" that returns 404 now **fails** point 5b, where previously it passed on the strength of its own label.

Is the second hop worth it?

On this federation, so far, no. **A depth-2 run reports both depths**: two summary tables, two explanatory sections, a list of every cell whose verdict differs, and a table of which second-hop pages were fetched. Both tables come from the *same capture* — the shallow view is the deep evidence with the second-hop pages set aside, not a second run — so any difference is the extra hop and not the page changing in between.

Two depth-2 runs on 21 September, of 18 and 14 second-hop requests, each changed **zero** verdicts. That is a finding rather than a disappointment: the points still marked review turn on a judgement ("clearly state") or quantify over things no crawl enumerates ("all research resources offered by the Node"), and no amount of fetching settles either kind. It should not be assumed to hold for another federation or a later date.

5. Configuration and customisation

The node list — a YAML file

The landing pages you want checked live in a configuration file, and that is the intended way to change them. `nodes.yaml` sits at the repository root and currently lists **thirteen** nodes: `bbmri-eric`, `cern`, `eosc-cz`, `eosc-dto`, `data-terra`, `eosc-fi`, `panosc`, `eudat`, `egi`, `geant`, `ebrains`, `eosc-it`, `eosc-sk`.

```
nodes:
- id: bbmri-eric
```

```
name: BBMRI-ERIC
url: https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/
eosc_page: https://eosc.eu/building-the-eosc-federation/eosc-node-bbmri-eric/
```

- `id` — the short handle used by `--only` and by the `show` command, and the filename its evidence is stored under.
- `url` — must be the URL registered in the EOSC EU Node Contributors Dashboard (section 1.2, field 6, "Website address"), because that registered URL is what the checklist defines as the Node Landing Page.
- `eosc_page` — the node's dedicated entry on eosc.eu, which point 4 requires the landing page to link to. Supplying it means a point 4 failure names the exact URL that is missing, so the fix is a one-line edit rather than an investigation.

You can also point at an entirely different file without editing the default: `--nodes /path/to/my-nodes.yaml`, available on `collect`, `assess` and `run`. That is the clean way to keep a separate candidate-node list alongside the production one.

Changing a URL. BBMRI-ERIC's `url` changed on 25 September 2026, from <https://www.bbmri-eric.eu/eosc-node-bbmri-eric/> to <https://dev3.bbmri-eric.eu/eosc-node-bbmri-eric/>, and the entry carries a comment saying so. `assess` takes each node's URL from `nodes.yaml` and its verdicts from the evidence on disk. So a bare `assess` now heads BBMRI-ERIC's row with the new address above verdicts taken from the old page. It says so: it compares the configured URL with the `requested_url` in the evidence, prints a warning, puts an **Evidence from a different URL** banner in `results.md` and `index.html`, and records the pair under `url_mismatch` in `results.json`. Until the published run is replaced by a new reviewed one, rebuild it with the node list it was collected with. That gives no warning, and reproduces the committed `results/` exactly, apart from the generation time:

```
git show 53081f6:nodes.yaml > /tmp/nodes-2026-09-24.yaml
uv run basic-check assess --run live-2026-09-24-no-italy --skip Italy --nodes /tmp/nodes-2026-09-24.yaml
```

The new address also behaves differently. On a trial run on 25 September its `robots.txt` read `User-agent: * / Disallow: /`. The tool honours that, so no page was requested and all ten points were `ERROR`. Section 6 of the [run guide](#) explains what `ERROR` does and does not mean. The step-by-step procedure for a URL change, through to new published results, is example 2 in section 11.

Adding a node

Adding a node touches **three** files, and the suite fails if you stop after the first.

File	What to add	Enforced by
<code>nodes.yaml</code>	The <code>id</code> , <code>name</code> , <code>url</code> and <code>eosc_page</code> block	<code>test_nodes.py</code> — required fields, unique id, well-formed URL
<code>checklist/approved-names.txt</code>	The node's official Tripartite-approved name	Point 3 cannot match a name that is not in the list
<code>checklist/approved-names-scoped.txt</code>	The same name, bound to the node id	<code>test_nodes.py</code> — every configured node's name must be covered

Then collect evidence for it before the next full `assess`, because `assess` exits 2 when a configured node has no evidence — correctly, since an unassessed node must not pass silently:

```
uv run basic-check collect --only <new-id> --depth 1 --delay 2.0 --results /tmp/scratch
```

Use a scratch `--results` directory while you are trying a node out. `results/` holds the reviewed, published run, and a full `assess` rewrites it. Example 1 in section 11 goes from these edits to a published run that includes the new node.

The approved-names list

Since the list became a committed default, point 3 has a name list **unless you turn it off**. Three flags govern it:

Flag	Effect
<i>(none)</i>	Uses the committed <code>checklist/approved-names.txt</code> . Its SHA-256 is recorded in every report
<code>--approved-names <file></code>	Replaces the default with your own list
<code>--no-approved-names</code>	Uses no list at all, not even the default — the pre-default behaviour
<code>--strict-separators</code>	Matches the separator glyphs literally. By default an approved name written with a vertical bar also matches a page that writes it with a dash or a plain space

The separator tolerance is the reason `test_names.py` is the largest file in the suite: a matcher loose enough to accept the punctuation variants real pages use is also loose enough to match the wrong thing, so the boundary is pinned from both sides.

A matched name is not a verified name. The unscoped list does not say which name belongs to which node, so a match establishes only that *some* approved name appears on the page. The evidence line says so explicitly. `approved-names-scoped.txt` binds names to node ids and removes that ambiguity; the report records which of the two was used.

Ad hoc URLs — straight from the command line

You can check a page without touching configuration at all. `--url` is repeatable:

```
uv run basic-check run --url https://example.org/our-eosc-node/
uv run basic-check run --url https://a.example/ --url https://b.example/
uv run basic-check show example-org-our-eosc-node --one-off
```

Point 4 needs to know which `eosc.eu` page the node ought to link to. Without it the point is still decided — a link to the federation index still fails — but the message cannot name the missing URL. Supply it for a single URL with `--eosc-page`:

```
uv run basic-check run --url https://eosc.panosc.eu/ \
--eosc-page https://eosc.eu/building-the-eosc-federation/eosc-node-panosc/
```

Two safeguards worth knowing about. A `--url` run writes to `results/one-off/` rather than `results/`, because `assess()` rewrites `results.md`, `index.html` and `results.json` wholesale — a shared directory would replace the published report with a one-row table. And ad hoc reports are titled "Ad hoc page check (not a federation run)" with a marker at the top, since the layout is otherwise identical to the real report and a stray single-node file that looked official would be a liability while a production decision is pending.

`--url` and `--only` are rejected together rather than one being quietly ignored: `--only` filters ids in the nodes file, and an ad hoc URL has no id there. `--skip` is rejected with `--url` for the same reason. Node ids are derived from host *and* path, so two pages on one host cannot overwrite each other's evidence.

✓ **`--only` no longer shrinks the report — the defect described in the previous edition is fixed.** It previously scoped the *report* as well as the fetching, so `assess --only data-terra` left `results/results.json` containing a single node while `index.html` still looked like a federation report. `--only` now narrows **which sites are contacted**, not what the report covers: a subset run refetches only the nodes you named, then rebuilds every row from the evidence already on disk, and records the selection so a reader can see the report is of mixed freshness. Four tests in `test_cli.py` pin this, including one that runs a real subset `assess` against the committed evidence and asserts the published directory still has every row.

`--skip` leaves nodes out, and the report says so. `collect`, `assess` and `run` accept `--skip`, followed by node ids or names. Names are matched case-insensitively, including the short form, so `Italy` names "EOSC Node Italy". Values can be comma-separated or the option repeated. Skipped nodes are not fetched, not assessed and not reported. They do not count as missing evidence, so `assess` does not exit 2 for them. The report is shorter as a result, so it states the omission: a "Skipped by request" banner in `results.md` and `index.html`, and a `skipped` list in `results.json`. Sixteen cases in `test_cli.py` pin this behaviour:

- both spellings (comma-separated and repeated) and all three forms of a name;
- that no request is sent to a skipped node by `collect` or `run`;
- that a skipped node without evidence does not trigger exit 2;
- that both reports carry the banner, and a run without `--skip` carries none;
- that a value matching no node is rejected, as are a value matching several nodes, skipping every node, and `--skip` with `--url`.

Before the commit, nine deliberate bugs were introduced into the new code one at a time, and the suite caught all nine.

The five commands

Command	What it does
<code>collect</code>	Fetch each landing page with Chromium and save the evidence. Needs the network and a browser
<code>assess</code>	Apply the checklist to already-collected evidence and write the reports. Offline, repeatable
<code>run</code>	<code>collect</code> , then <code>assess</code> , in one invocation
<code>points</code>	Print the checklist, its provenance, and which function decides each point
<code>show</code>	Print one node's results in the terminal

Only `collect` and `run` need a browser or network access. `points`, `assess`, `show` and the whole test suite need neither.

Command line options

Option	Available on	Default	Effect
<code>--nodes / -n</code>	<code>collect</code> <code>assess</code> <code>run</code>	<code>nodes.yaml</code>	Which node list to read

Option	Available on	Default	Effect
<code>--url</code>	collect assess run	—	Check this URL directly, without configuration. Repeatable
<code>--eosc-page</code>	collect assess run	—	With a single <code>--url</code> : the node's own eoscc.eu page, for point 4
<code>--results</code>	collect assess run show	results/	Output directory (results/one-off/ for <code>--url</code> runs)
<code>--only</code>	collect assess run	all	Comma-separated node ids. Narrows which sites are contacted, not what the report covers
<code>--skip</code>	collect assess run	none	Node ids or names, comma-separated or repeated. Left out of fetch and report; the report says so
<code>--depth</code>	collect run	1	0 = landing page only; 1 = one hop; 2 = two hops, reporting both depths. Range-checked 0–2
<code>--fetch-budget</code>	collect run	60	Depth 2 only: run-wide ceiling on second-hop requests
<code>--max-children</code>	collect	8	Cap on followed pages per node
<code>--delay</code>	collect run	2.0	Seconds between hosts
<code>--checklist / -c</code>	assess run points	checklist/v3.0.yaml	Which checklist version to apply
<code>--approved-names</code>	assess run	committed default	Text file of Tripartite-approved names, one per line, replacing the committed default
<code>--no-approved-names</code>	assess run	false	Use no name list at all, not even the committed default
<code>--strict-separators</code>	assess run	false	Match the separator glyphs in an approved name literally
<code>--run</code>	assess run	timestamp	Label for the run, recorded in every report
<code>--one-off</code>	show	false	Read results/one-off/ instead of results/

Note that `--max-children` is on `collect` but not on `run`, so a combined run uses the default cap of 8. Use the two commands separately if you need to change it.

Exit codes

Code	Meaning
0	The run completed. This does NOT mean every node passed — FAIL verdicts still exit 0
2	Bad usage, or a configured node produced no evidence at all

A non-zero exit means the tooling failed, not that a node did. The only non-zero exit from a completed assessment is missing evidence. A node that violates six checklist points still exits 0, because a compliance finding is the output, not an error. So in CI a red cross means something broke; it never means "a node is non-compliant". Read the matrix, not the exit status.

The reference checklist — the single configuration point

`checklist/v3.0.yaml` is data, not code, and it is the one place the reference checklist is configured. The tool never downloads the checklist document, so there is no URL to change when it is revised: the document is committed next to the YAML. One line in `src/basic_check/cli.py` selects the default revision, and `--checklist / -c` on `assess`, `run` and `points` overrides it for one run:

```
DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.0.yaml"
```

Nothing else in the codebase names the file. The help texts, the name of the generated `checklist-v3.0.html` and the tests in `test_checklist.py` all derive from that line.

```
checklist/  
v3.0.yaml           # the checklist as data  
20260910_Node_Landing_Page_..._v3.0.pdf # the document it came from  
README.md          # the revision procedure
```

Each point carries its verbatim requirement text, a `decidable` flag, a `decidable_note` explaining how the tool treats it and why, and `implemented_by` naming the function that decides it. The file header records provenance:

```
checklist_version: "3.0"  
checklist_date: "2026-09-15"  
source_document: 20260910_Node_Landing_Page_Verification_Checklist_v3.0.docx  
source_file: 20260910_Node_Landing_Page_Verification_Checklist_v3.0.pdf  
source_sha256: 31e0acbc0c40eee271213ac6f2dcc1ef634c7b067119f515ddb06ddbc0145b59
```

Three guarantees are enforced by tests rather than by good intentions.

- **The source document is committed and pinned.** A transcription you cannot audit against its source is a rumour. If the document is ever replaced under the same filename, the hash test fails and forces a deliberate decision rather than a silent one.
- **Every point maps to one rule, and every rule to one point.** A check no point claims is either dead code or a rule the reports never explain. A point naming a function that does not exist fails immediately.

- **A revision must be a new file.** The declared `checklist_version` must match the filename, so v3.1 cannot be edited into `v3.0.yaml`. Reports name the file they were generated from, so editing in place would retroactively invalidate every past run.

What the hash does and does not prove. The committed file is a PDF rendering of the `.docx` named in `source_document`, not that `.docx` itself. It pins the exact bytes the transcription was made from — which is what catches a document being silently swapped — but it is not a signature over the authoritative original. Replacing the PDF with the real `.docx` is a two-line change in the YAML plus a re-hash.

What the YAML governs is the **reporting surface**: titles, the matrix columns and their order, the quoted wording, the decidable/review framing. What it does *not* govern is the logic — `checks.run_all()` calls ten hardcoded functions and never reads the file. The YAML declares the checklist; the Python interprets it. That is precisely why `implemented_by` and its tests exist.

Adding a newer checklist revision

```
# 1 - commit the new source document into checklist/
# 2 - copy the YAML to the new version, never edit in place
cp checklist/v3.0.yaml checklist/v3.1.yaml
shasum -a 256 checklist/<new-source-document> # macOS spelling

# 3 - update checklist_version, checklist_date, source_document,
#       source_file and source_sha256 in checklist/v3.1.yaml
# 4 - re-read every point against the new document and fix any
#       check_* function whose requirement changed    <-- manual step
# 5 - try it offline against the saved evidence, before switching
uv run basic-check assess -c checklist/v3.1.yaml --results /tmp/v31 # after copying results/evidence there
# 6 - make it the default: in src/basic_check/cli.py
#       DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.1.yaml"

uv run pytest -q # consistency
uv run basic-check run --results /tmp/new-run # then review and publish
```

Example 3 in section 11 gives every command, including publication and the removal of the old `results/checklist-v3.0.html`. Step 4 is the one the tooling cannot do for you. The tests confirm that the checklist and the code agree about *which* rules exist; they cannot confirm that a rule still means what the revised document says. `checklist/README.md` repeats this procedure inside the repository.

Personal data in the evidence and the reports

Evidence files and all four report formats are written with personal data masked by `src/basic_check/privacy.py`. A personal email address keeps only its domain (`XXXXXX@example.org`), and a phone number keeps only its international prefix (`+31 XXXXXX`). A labelled national number such as `tel. (09) 123 4567` becomes `tel. XXXXXX`, and a number whose spaces are written `%20` inside a `tel:` link is masked too. A name is masked when it appears next to a masked address. Role mailboxes such as `support@`, `info@` and `it@helpdesk....` are kept, because point 6 depends on them. Nothing about this is configurable: a flag to turn it off would be a way to publish personal data by mistake. The rules and their limits are in section 6 of the [run guide](#) ("Personal data is masked"), and `tests/test_privacy.py` fixes both sides: what must be masked, and what must not be. One test reads the committed `results/evidence/` itself and fails if any file contains a personal address or a phone number, so unmasked evidence cannot be committed unnoticed. It was confirmed to bite: restoring one unmasked evidence file makes it fail.

Constants that are not yet flags

Several limits are module constants rather than command line options: in `fetch.py`, `MAX_PER_PURPOSE = 2`, `MAX_PER_PURPOSE_D2 = 1`, `CHILD_DELAY_S = 1.2`, `GRANDCHILD_DELAY_S = 1.5` and the navigation timeouts; in `checks.py`, the render gate's thresholds `DOM_ELEMENT_FLOOR = 10` and `EMPTY_DOCUMENT_CHARS = 500`. They can be promoted to CLI options if you need to vary them per run.

Two commands, on purpose

`collect` and `assess` are deliberately separate. Assessment re-runs offline against saved evidence in `results/evidence/`, so changing a rule never means re-requesting the nodes' pages. `assess` also exits non-zero when a configured node has no evidence, so a short table cannot quietly look complete.

6. Continuous integration, and running it from the GitHub web interface

There are two workflows, and the distinction between them matters.

Workflow	Trigger	What it does
<code>tests.yml</code>	Every push and pull request, plus manual dispatch	ruff, then pytest, then publishes the JUnit report to GitHub Checks. No browser, no network
<code>compliance.yml</code>	Manual dispatch only — deliberately no schedule	Runs the tests, installs Chromium, then checks the real landing pages and publishes a counts summary plus the full reports as an artifact

The test workflow runs four steps: `uv sync --locked`, `ruff check src tests`, `pytest -q --junitxml=results/junit.xml`, then publication of the JUnit XML to GitHub Checks. Chromium is not installed and does not need to be, because no test opens a browser.

`--locked` matters more than it looks. It installs exactly what `uv.lock` pins and **fails** if the lockfile has drifted from `pyproject.toml`. Plain `uv sync` would silently re-resolve, so a dependency change could land with a green tick while CI tested a different set of versions than the one committed. The compliance workflow uses `--locked` for the same reason with sharper stakes: a silently re-resolved Playwright, selectolax or lingua can change what a page looks like to the tool, and the output of that workflow is a set of verdicts about named organisations.

A habit worth keeping. A green tick means "nothing objected", not "everything was verified". A suite that collects zero tests also passes. The test count in the CI log is the thing to read — it currently says "339 passed".

Running the compliance scan from the browser, with no Terminal at all

This is the whole point of the second workflow: the node scan can be launched from the GitHub web interface without a command line anywhere.

- Open the **Actions** tab of the repository.
- Click **compliance run** in the left-hand sidebar.
- Click the **Run workflow** dropdown on the right of the run list.

- Leave the three fields at their defaults for every configured node at depth 1, or fill them in.
- Click the green **Run workflow** button, then refresh.
- Open the run. A **counts summary** is printed on the run page itself, under the job list. The per-node matrix and evidence are in the [compliance-results](#) artifact attached to the run.

Input	Default	Meaning
only	empty = every configured node	Comma-separated node ids, e.g. egi,eudat
depth	1	1 follows one hop of checklist-relevant links; 0 checks the landing page only; 2 adds a second hop
delay	2.0	Seconds between nodes

The web form now reaches depth 2. The [depth](#) input is a [choice](#) offering 1, 0 and 2, so the dual-depth report is available from the Actions tab as well as the command line. The previous edition recorded this as a limitation.

Why the run page shows counts and not the matrix. This is a deliberate change since the previous edition. The per-node verdicts are already published in the committed [results/results.md](#), so repeating them leaks nothing new — but a workflow run is *unreviewed output*, and a world-readable page of FAIL verdicts against named organisations, produced automatically and carrying the repository's name, reads as a finding rather than as a draft. The summary therefore carries counts, an explicit "unreviewed automated output" warning, and a pointer to the artifact. It also refuses to summarise [results/results.json](#) unless the check step actually succeeded — otherwise a failed run would publish the last *reviewed* run's counts under this run's number, which is the most misleading thing the workflow could do.

The equivalent from a Mac Terminal, if you have the GitHub CLI installed, is:

```
gh workflow run compliance.yml --ref main
gh workflow run compliance.yml --ref main -f only=egi,eudat -f depth=0
gh run list --workflow compliance.yml --limit 3
gh run view <run-id> --log
```

Who can launch it, and who can read the results. Triggering a workflow requires write access to the repository, so nobody browsing a public repo can start a run — the Run workflow button is not shown to them. The compliance workflow is also [workflow_dispatch](#) only, with no schedule and no [pull_request](#) trigger, so no fork or pull request can cause it to send requests to the nodes' websites. Note separately that on a public repository the committed reports under [results/](#) are readable by anyone, and the compliance workflow writes the results table into the run summary, which is public too. Both are publication decisions rather than technical ones.

Two design choices in that workflow are worth knowing. It runs pytest before installing Chromium, so a broken rule set cannot send thirty-odd requests to real sites to produce wrong verdicts. And a concurrency group prevents two runs overlapping onto the nodes. Expect three to four minutes, most of it the Chromium download rather than the requests.

One caveat about CI results. GitHub runners use datacenter IP addresses, which are more likely to meet a bot wall than a home or office connection. A 403 from a runner is evidence about the runner, not about the node.

Correcting the previous edition. It stated that the CI run of 18 September "reproduced the local figures exactly, 29 PASS / 6 FAIL / 55 MANUAL_REVIEW". Checking the actual run logs while preparing this edition, that does not hold. There were two compliance runs that day and they disagreed with each other and with the committed results:

18 September run	Source	Tally
13:31 UTC	CI, run 35350733094	29 / 6 / 55
committed results.json	local, commit 062aca3	26 / 6 / 58
14:39 UTC	CI, run 35357569881	24 / 5 / 61

So 29 / 6 / 55 was a real CI figure, but it was not a reproduction of the committed local run, and an hour later the same workflow produced a third answer — because GÉANT already returned **HTTP 403 to the runner on 18 September**, logged as [\[8/9\] geant HTTP 403](#). The claim of exact cross-network agreement was wrong.

Treat any single tally as a reading, not a constant. Three runs in one afternoon gave three different numbers without a line of rule code changing between them. What is stable is the *finding* (see section 10), not the count. Any figure quoted in a presentation should name the run it came from.

7. Tests that exist because of real defects

These are worth reading as documentation of what the tool got wrong, and each carries its reasoning in the test itself rather than only an assertion. Five are described below, followed by the defects fixed since the previous edition and one found while preparing this one.

[test_a_well_rendered_page_still_fails_when_the_link_is_genuinely_absent](#) Reporting "no contact route of any kind was found" from a page that had not rendered. That describes the tool, not the node. Absence-based failures are now gated on the page having actually rendered — and this test stops the gate becoming a blanket excuse.

[test_point4_rejects_domains_that_merely_end_in_eosc_eu](#) A domain check written as `endswith("eosc.eu")` also matched `myeosc.eu` and `not-eosc.eu`. Found by probing the function directly, not by the tests.

[test_point1_does_not_fail_on_403_with_no_login_offered](#) Originally asserted FAIL. Inverted after GÉANT served HTTP 200 and then 403 once link following made a few more requests to the same host. Nothing about the site had changed; the tool had become more annoying. Turning that into "not publicly accessible" would be the checker blaming a node for its own request volume. This has since paid for itself — see section 10.

[test_every_link_a_check_can_use_is_a_link_the_crawler_will_follow](#) The crawler looked for "acceptable use" while the check also accepted "terms of use", so links the checks relied on were never fetched. Four PASSes rested on a link label while the document sat one request away. Nothing failed; the output was quietly weaker than it claimed.

[test_run_does_not_pass_typer_descriptors_to_its_helpers](#) `basic-check run` was documented as working and crashed on every invocation: calling a Typer-decorated function from Python passes its option descriptors, not their values. Nothing tested the CLI, so 53 tests stayed green throughout.

Two patterns recur in that list and are worth carrying into the wider assessment work. First, **a passing suite is evidence about the tests, not about the tool** — three of these five defects were found by probing behaviour,

not by a failing test. Second, **the most dangerous failure mode here is not a wrong verdict but an overstated one**: output that reads as verified when the underlying collection was incomplete.

The two latent defects are now fixed — and there were six, not two

The previous edition recorded two latent defects in `report.py`: a vertical bar in a node **name** producing a misaligned Markdown row, and a newline inside an evidence string breaking the Markdown list. Neither was triggered by the node list at the time. Both are fixed at commit [014682c](#).

The fix is a boundary rather than a patch at each site. Two helpers now sit where values enter Markdown:

Helper	Does	Used for
<code>_md_text(value)</code>	Collapses all whitespace, so a newline cannot break the structure	Headings, list items, bold runs
<code>_md_cell(value)</code>	<code>_md_text</code> , then escapes the vertical bar	Table cells

Writing the tests first was what made the fix honest. Ten tests were written before any code changed, and **eight of them failed**, as intended. One of the two that passed did so by design: it guards against over-fixing, asserting that collapsing whitespace does not mangle ordinary evidence text.

Applying the two helpers at every boundary then revealed four more instances of the same bug class than the two that had been reported: the depth-change table, the followed-link reason, a point title reaching the `COLUMN_GLOSS` fallback, and a newline in a reviewer action. The reported defects were the two someone had happened to notice; the class was wider.

Two details are worth keeping:

- **Pipes in URLs stay percent-encoded as %7C**. Inside a Markdown autolink a backslash is not an escape character, so escaping would have produced a visibly broken link.
- **Regenerating the published report from the committed evidence changes no cell**. That was checked rather than assumed: the fix is presentational, and the verdicts are unaffected.

A test that counts the wrong thing passes for the wrong reason. The first version of these tests counted raw vertical bars in a rendered row, which an escaped bar also satisfies. The assertions were wrong, not the code. They now count *unescaped* delimiters via a small `_columns(row)` helper, and the escaped output was additionally checked against GitHub's own Markdown renderer rather than trusted to local reasoning.

A URL change relabelled old evidence, silently — fixed on 26 September 2026

`assess` heads each row with the URL in `nodes.yaml` and takes the verdicts from the evidence on disk. After BBMRI-ERIC's URL changed on 25 September, a bare `assess` would have printed the new `dev3` address above verdicts about the old page, and nothing in the output said so. The address really fetched was only in the evidence file.

`assess` now compares each node's configured URL with the `requested_url` its evidence records. A difference produces a warning on the terminal, an **Evidence from a different URL** banner in `results.md` and `index.html`, and a `url_mismatch` list in `results.json`. Two tests in `test_cli.py` pin both directions: a changed URL is flagged in all three places, and the published run rebuilt with the node list it was collected with gains no banner and no new key. The exit code is unchanged, because the table is complete. Example 2 in section 11 shows how to clear the warning.

A defect found while preparing this edition, since fixed

`check_3` returned a fixed sentence stating that "the tool has no authoritative list of approved names" — **even when a list was supplied and a name matched**. The name finding was appended correctly to the evidence lines, so a single point 3 result could contain both a match and a claim that no list existed. In the published run this affects BBMRI-ERIC and EUDAT, the two nodes whose names matched.

The verdict was never affected: point 3 is `MANUAL_REVIEW` in either case. But this is precisely the failure mode this section opens with — output whose prose is weaker or stranger than the evidence behind it.

The cause was two independent writers. The summary sentence was a literal string in `check_3`, while the evidence line was computed by `_approved_name_note` from the list and the page. Nothing tied them together, so they drifted. The fix is not a reworded sentence: `_approved_name_note` now returns a `_NameNote` carrying both a `state` and its evidence line, and the summary clause is looked up from that same state. A summary that contradicts its own evidence would now require the state to contradict itself.

The six states, and what each tells a reader of the summary alone:

State	The name half of point 3
<code>absent</code>	no list was supplied, so the name was not checked
<code>unscoped-for-node</code>	the list holds no approved name for this node
<code>no-body</code>	names were supplied, but no page body was captured, so the name could not be looked for
<code>not-found</code>	looked for and not found in the body — not proof of absence, since the <code><title></code> is not searched
<code>matched</code>	a name written against this node was found in the body
<code>matched-unscoped</code>	a name was found, but the list is unscoped and does not say which node it belongs to

Twelve cases pin this — nine test functions, one of them parametrized across all four reachable supplied-list states — asserting that the summary never denies having a list it was given. The reviewer action adapts too: it no longer asks a reviewer to check the name against a list the tool has already matched it against.

Re-running the nine published nodes confirms the blast radius is prose only. Verdicts and evidence lines are byte-identical before and after — the tally is 28 PASS / 7 FAIL / 55 `MANUAL_REVIEW` either way. Only `message` and `reviewer_action` changed. BBMRI-ERIC and EUDAT now read "an approved name was found in the page body, but the list supplied is unscoped, so it does not say which node the name belongs to" in place of the denial.

One asymmetry is left deliberately. When no EOSC image asset is found, the summary discusses only the logo gap and says nothing about the name, while the evidence line still reports the name state. That summary makes no false claim, so it was not rewritten; EGI, GÉANT and EBRAINS take that branch in the published run.

8. Running it

```
uv sync
uv run playwright install chromium --with-deps
```

```

uv run basic-check points    # checklist, provenance, rule mapping
uv run basic-check collect  # landing page + up to 2 bounded hops
uv run basic-check assess   # evidence -> reports, offline, repeatable
uv run basic-check run      # collect, then assess
uv run basic-check show egi # one node in the terminal

uv run pytest -q            # 339 cases, ~5-6 s, no network, no browser
uv run ruff check src tests

```

A few combinations that answer most real questions:

```

# two nodes only, landing pages alone, no link following
uv run basic-check run --only egi,eudat --depth 0

# the deeper run, reporting both depths side by side
uv run basic-check run --depth 2
uv run basic-check run --depth 2 --fetch-budget 20 # a stricter ceiling

# gentler on the nodes: 5s between hosts, at most 4 pages each
uv run basic-check collect --delay 5 --max-children 4
uv run basic-check assess --run gentle-run

# re-score saved evidence after editing a rule - no requests at all
uv run basic-check assess

# apply a different checklist revision
uv run basic-check assess -c checklist/v3.1.yaml

# point 3 uses the committed name list by default; override or disable it
uv run basic-check assess --approved-names ~/Documents/my-names.txt
uv run basic-check assess --no-approved-names
uv run basic-check assess --strict-separators

# a single page, without touching nodes.yaml
uv run basic-check run --url https://example.org/our-node/ \
  --eosc-page https://eosc.eu/building-the-eosc-federation/eosc-node-x/
uv run basic-check show example-org-our-node --one-off

# a separate candidate list, written elsewhere
uv run basic-check run -n ~/Documents/candidate-nodes.yaml \
  --results ~/Documents/candidate-results

```

Outputs land in `results/`: `results.md` (renders with colour directly on GitHub), `index.html`, `checklist-v3.0.html`, `results.csv`, `results.json`, and per-node evidence under `results/evidence/`.

Use a scratch output directory for anything exploratory. `uv run basic-check assess` rewrites `results/` wholesale, and `results/` holds the reviewed run that the published report is built from. Pass `--results /tmp/scratch` for trial runs, and check git status `--short results/` before committing.

A fuller treatment — installation, configuration, reading the output, troubleshooting — is in the [installation, configuration and run guide](#).

9. Setting it up on macOS

Only two things need to be installed on the Mac: [git](#) and [uv](#). Everything else — the correct Python, every runtime dependency, `pytest` and `ruff` — is installed by `uv sync` into a project-local `.venv/`. There is no need to install Python packages by hand, and nothing is installed system-wide.

Prerequisite	How to get it	Notes
git	Xcode Command Line Tools	Running <code>git --version</code> on a clean Mac triggers the install prompt. Or: <code>xcode-select --install</code>
uv	Installer script, or Homebrew	<code>brew install uv</code> , or the curl installer shown in step 2 below
Python 3.12+	Installed by uv	<code>uv sync</code> downloads a suitable Python if the system one is too old. macOS system Python is not used
Chromium	<code>uv run playwright install chromium</code>	Only needed to check real pages. Not needed to run the tests

The tests need no browser and no network. `playwright install chromium` fetches three components — Chromium, the headless shell and FFmpeg — totalling roughly **660 MB**, not the ~150 MB stated in the previous edition of this document. The cache at `~/.cache/ms-playwright` is shared by every project on the machine, so a second checkout downloads nothing. It is only required by the `collect` step, which visits real node pages. If you only want to run the suite, skip it — exactly as the CI job does, which never installs a browser.

The exact command sequence

Each block can be pasted as-is into Terminal. Lines beginning with `#` are comments.

```
# 1 - confirm git is present (prompts to install if it is not)
git --version

# 2 - install uv, then make it visible to the current shell
curl -LsSf https://astral.sh/uv/install.sh | sh
exec $SHELL -l
uv --version

# 3 - clone the repository and enter it
cd ~/Documents
git clone https://github.com/marioreale/eosc-basic-compliance.git
cd eosc-basic-compliance

# 4 - confirm you have what you expect
git status
git log --oneline -5

# 5 - create the environment and install every dependency
uv sync

# 6 - run the suite: 339 cases, ~5-6 s, no network, no browser
```

```
uv run pytest -q
uv run ruff check src tests
```

At that point the test suite is running locally. The next two commands are only needed if you want to check real landing pages rather than run the tests.

```
# 7 - install the browser used to render pages
uv run playwright install chromium

# 8 - check one page without touching nodes.yaml
uv run basic-check run --url https://eosc.panosc.eu/ \
--eosc-page https://eosc.eu/building-the-eosc-federation/eosc-node-panosc/

# 9 - or run the full federation list from nodes.yaml
uv run basic-check run
open results/index.html
```

Keeping it up to date, and pushing changes back

```
# pull later changes
git pull
uv sync          # in case dependencies changed

# commit and push your own edits (you own the repository)
git add -A
git commit -m "Describe the change"
git push
```

The clone above uses HTTPS, so the first `git push` will ask for credentials. GitHub no longer accepts account passwords here: use a personal access token as the password, or install the GitHub CLI and run `gh auth login`, which configures the credential helper for you. An SSH key works equally well if you clone the `git@github.com`: URL instead.

The reproducibility caveat in the previous edition is resolved. `uv.lock` is now committed and no longer in `.gitignore`. `uv sync` on your Mac installs the same resolved versions CI uses, and both workflows install with `uv sync --locked`, which *fails* rather than re-resolving if the lockfile has drifted from `pyproject.toml`. Dependency versions are now part of what the repository pins, alongside the checklist source hash and the approved-names hash.

10. The published figures

Current run: 24 September 2026

From `results/results.json`, run `live-2026-09-24-no-italy`, collected live on 24 September 2026 at `--depth 1` and assessed against the official unscoped names list (13 names, SHA-256 `871161a5...`). Twelve nodes were assessed, and EOSC Node Italy was skipped with `--skip Italy` because `eosc.it` had no address record. The run made 44 requests (12 landing pages and 32 child pages), and all 12 landing pages returned HTTP 200. The figures below are from the re-assessment at commit `29dead8`, made from the same evidence after the point 6 and 5b/5c fixes. The first assessment gave 45 / 6 / 69.

Masked on 25 September 2026. Commit `ada1b4a` masked the personal data in `results/evidence/` and rebuilt the reports offline, with the node list the run was collected with (from commit `53081f6`), so BBMRI-ERIC is still shown against the `www.` address its evidence came from. All 120

verdicts and the tally below are unchanged. The only other differences are the generation time and a few character counts, which drop by the length of the masked text. The unmasked versions remain in the git history.

Verdict	Cells
 PASS	44
 FAIL	6
 MANUAL_REVIEW	70
Total	120 (12 nodes × 10 points)

All six FAILs are point 4. Four landing pages have no eosc.eu link at all (Data Terra, EOSC Finland, EGI, EBRAINS), and two link only to the federation index (PaNOSC, GÉANT). Compared with the run below, EOSC DTO moved from [FAIL](#) to [PASS](#) on points 4 and 6. GÉANT was served this time, which settled points 1, 6 and 7 as [PASS](#) and point 4 as [FAIL](#). CERN, Czechia and Slovakia appear for the first time. The point 6 and 5b/5c fixes account for the other two differences: GÉANT 5b is now [PASS](#), and EBRAINS 6, which passed on 21 September, is now [MANUAL_REVIEW](#). No other cell changed.

The run was reviewed by hand before publication. The review is in [results/REVIEW-2026-09-24.md](#), and it leaves the tool's output untouched. It found three point 6 PASSes that rest on link text alone: Czechia's "National Support" is a funding page, EBRAINS's is a EuroHPC proposal service, and BBMRI-ERIC's is a service overview. BBMRI-ERIC's PASS was later upheld, because its contact page lists helpdesk mailboxes. It also found one AUP link the tool missed, on GÉANT, whose link text is prose. Those limitations were recorded there and have since been fixed in the checks; see "Point 6 and 5b/5c after the review" below. The committed [results/](#) was then re-assessed with the fixed checks from the same evidence. The review also records that the configured URLs for CERN (a sign-in form) and EBRAINS (the general homepage) are not descriptive landing pages, and both rows are published with that caveat.

[test_names.py::test_the_official_names_match_the_nodes_that_show_them](#) is pinned to the committed evidence. It now expects five matching nodes (BBMRI-ERIC, Czechia, EUDAT, GÉANT, Slovakia) in place of two. Re-measured on 25 September against the masked evidence: the node-scoped list finds the same five, and [--strict-separators](#) finds none of the twelve, because every one of the five writes the name with a different separator from the official list.

Since the published run

A trial run on 25 September 2026, with BBMRI-ERIC's new URL, is **not published** and has not been reviewed. Two things it showed concern the tool rather than the nodes, and are recorded here for whoever runs the next collection:

- BBMRI-ERIC's [dev3](#). address disallows every path in [robots.txt](#), so all ten of its points were [ERROR](#). The next published run needs either the public address or an exception for the checker.
- GÉANT answered HTTP 403 with the Cloudflare challenge again, and its decided cells moved to review, as on 21 September.

Point 6 and 5b/5c after the review

The review of the 24 September run found four tool issues. The three below are fixed in the checks. The fourth, noting the accepted name separators next to each point 3 match, is still open. The fixes change the assessment only, so the committed evidence was re-assessed offline and no node was contacted again.

- **Point 6 passed on link text alone.** Any link whose label contained "support" was a PASS. Now a bare "support" link is still followed but settles nothing. A PASS needs one of three things: a label or address that names a helpdesk, service desk, ticket system, support team or request; a helpdesk host ([hd.](#), [support.](#), [helpdesk.](#)) or mailbox ([support@](#), [it@helpdesk....](#)); or a followed page that names a helpdesk, service desk or ticket system, or gives such an address. Page prose about "support" no longer counts, because EBRAINS's EuroHPC proposal page says "Technical Support" and "Application Support team". Every followed page is now read, where before only the first was. BBMRI-ERIC's helpdesk mailboxes are on the second.
- **5b/5c ignored words spelled with hyphens in the address.** GÉANT links its AUP with a sentence as the label, and the words appear only in [/geant-node-acceptable-use-policy/](#). Link matching now also reads the address with its separators turned into spaces. The crawler and the checks share that matcher ([patterns.link_haystack](#)), so a future collection will fetch the page as well.
- **The 5b/5c hint said "re-run with --depth 1" in every case.** Now it gives the actual reason the target was not fetched: the run did not follow links, the target is a PDF, a cap was reached, the target is on another site, or it was not selected when the evidence was collected. It suggests a re-run only when one would help.

Re-assessing the committed evidence with the fixed checks gives 44 PASS, 6 FAIL and 70 MANUAL_REVIEW, against 45/6/69 as first published. Exactly three cells change:

Node	Point	First assessment	Re-assessment (published)	Review determination
EOSC Node Czechia	6	PASS	MANUAL_REVIEW	not met
EBRAINS	6	PASS	MANUAL_REVIEW	review
GÉANT	5b	MANUAL_REVIEW	PASS (pointer)	met as a pointer

BBMRI-ERIC's point 6 stays PASS, now on the helpdesk mailboxes its contact page lists ([it@helpdesk.bbmri-eric.eu](#), [elsi-helpdesk@...](#), [rd@helpdesk....](#)) rather than on the "Services & Support" label. The review first proposed "review", considering only [contact@bbmri-eric.eu](#). It was revised to "met" on this evidence.

Previous run: 21 September 2026

The rest of this section is kept as the record of the earlier published run. It no longer describes [results/](#), which the run above replaced; that run's files are in the git history at [47f08af](#).

From [results/results.json](#) as it stood at [47f08af](#), run [2026-09-21-1738](#), checklist v3.0 of 15 September 2026.

Two timestamps matter here and they are not the same. The **evidence was collected** on 21 September between 13:04 and 13:07 UTC. The **report was regenerated** from that same evidence at 17:38 UTC, after the name-matching and report-scoping work landed. No node was contacted again in between — which is the point of keeping [collect](#) and [assess](#) separate.

Nine nodes, not thirteen. [nodes.yaml](#) configures thirteen nodes, but the published report covers the original **nine**. CERN, EOSC Node Czechia, EOSC Node Italy and EOSC Node Slovakia have been added to the configuration and have not been collected into the reviewed run. [assess](#) names all four as missing evidence and exits 2 rather than quietly producing a nine-row table that looks complete.

Verdict	Cells
 PASS	28

Verdict	Cells
 FAIL	7
 MANUAL_REVIEW	55
Total	90 (9 published nodes × 10 points)

Eight of the nine landing pages answered HTTP 200; geant.org returned **HTTP 403**, redirecting to a Cloudflare bot-protection challenge, having served 200 at 12:09 the same afternoon after several runs.

Against the results committed on 18 September (26 / 6 / 58), **eleven cells changed**, from three separate causes — so this is not a single story about GÉANT:

Node	Cells	Change	Why
GÉANT	4	decided → review (point 1 PASS, 4 FAIL, 6 PASS, 7 PASS all now review)	HTTP 403. The tool declines to assess a page it cannot see
EUDAT	5	review → PASS (points 1, 4, 5b, 6, 7)	EUDAT returned HTTP 500 on 18 September and 200 now. The node was unreachable then, not non-compliant
EOSC DTO	2	review → FAIL (points 4, 6)	The render-gate fix in commit 5d347c7 : a consent banner no longer excuses a missing link

Two of those three causes are the *tool's own reach* changing, not the nodes changing. Only the EOSC DTO pair reflects a corrected rule.

That GÉANT row is the safeguard from section 7 working as designed: the tool declines to assess a node it cannot see rather than publishing a block as a compliance failure. It also means the published tally is not a like-for-like comparison with any earlier figure — one node is unassessed rather than compliant, and one node was unreachable last time.

The stable finding, across every run since 17 September, is point 4. Individual verdicts move with network conditions; the point-4 column does not. Six nodes fail it outright in this run — EOSC DTO, Data Terra, EOSC Finland, PaNOSC, EGI and EBRAINS — and GÉANT failed it on index-only grounds at 12:09 before the block, leaving BBMRI-ERIC and EUDAT as the only two that link correctly to their own eosc.eu page. That is the finding worth taking to a self-assessment discussion, not the tally.

The run was collected at [-depth 2](#), and **the depth-1 and depth-2 tallies are identical, cell for cell**: 28 / 7 / 55 both ways.

Node	HTTP	Pages followed	of which second-hop
BBMRI-ERIC	200	6	2
EOSC DTO (D4Science)	200	1	0

Node	HTTP	Pages followed	of which second-hop
Data Terra	200	1	0
EOSC Finland	200	4	1
PaNOSC	200	4	1
EUDAT	200	7	4
EGI	200	4	1
GÉANT	403	0	0
EBRAINS	200	9	5

Point 3 in the 21 September run, and a correction to the previous edition

The previous edition stated that point 3 was review for every node "because ~~--approved-names~~ was not supplied". That is **not** what the published run records. A name list *was* used — the committed default, whose SHA-256 the report pins — and the run metadata says so: `default_used: true`, nine names.

Point 3 is [MANUAL_REVIEW](#) for all nine nodes for a different and more durable reason: **the logo half of the point is a human judgement**. The checklist asks whether the EOSC logo is "clearly and visibly" shown, and no list of names settles that. What the name list changed is the *evidence*, not the verdict:

Outcome	Nodes
An approved name matched on the page	2 — BBMRI-ERIC, EUDAT
No approved name found in the page body	6 — EOSC DTO, Data Terra, EOSC Finland, PaNOSC, EGI, EBRAINS
Names supplied but no page body captured (HTTP 403)	1 — GÉANT

Three caveats the tool states itself, and which matter more than the count:

- The unscoped list does not bind a name to a node, so a match shows only that *some* approved name appears — not that it is this node's own name.
- The `<title>` element is not searched, only the page body. Several of the six "not found" results say so explicitly.
- Most of those six pages *do* contain the phrase "EOSC Node"; what is absent is the full approved form.

The current default list carries **thirteen** names, one per configured node, with a scoped variant that binds each name to its node id. The 21 September run predates that and used the nine-name version. The published run of 24 September used the thirteen-name list, and point 3 is [MANUAL_REVIEW](#) for all twelve of its nodes, for the same reason.

11. Worked examples

Three step-by-step procedures, one for each change that comes up most often: adding a node, changing a node's URL, and adopting a new revision of the checklist. Each lists which files to edit, the exact commands, and how to put the result into the published [results/](#) directory.

Four rules apply to all three:

- **Try it in a scratch folder first.** `--results /tmp/trial` writes nowhere near `results/`, so nothing published changes until you decide it should.
- **Run `uv run pytest -q` after every edit.** It needs no network, takes a few seconds, and names the file you got wrong.
- **Contact as few sites as possible.** Only `collect` and `run` send requests. `assess` works on saved evidence, so it can be repeated as often as you like.
- **A new published run needs a human review before it is committed.** The tool's verdicts are not a compliance statement (see "Reading the output" in the [run guide](#)). Write the review down, as `results/REVIEW-2026-09-24.md` does for the current run, and commit only then.

The run labels below (`live-2026-10-01-...`) are examples. Use the date of your own run.

Example 1 — adding a node and publishing it

The example adds a node with the placeholder id `eosc-example`. Replace every value with the real ones.

Step 1: edit three files. All three are at the repository root or in `checklist/`, and all are edited by hand.

`nodes.yaml`: add the entry. `url` is the landing page as registered in the EOSC EU Node Contributors Dashboard ("Website address", section 1.2, field 6). Copy `eosc_page` from the [federation index](#); do not guess it.

```
- id: eosc-example
  name: EOSC Node Example
  url: https://eosc-node.example.org/
  eosc_page: https://eosc.eu/building-the-eosc-federation/eosc-node-example/
```

`checklist/approved-names.txt`: add the approved name exactly as the Tripartite list writes it:

```
| EOSC Node | Example
```

`checklist/approved-names-scoped.txt`: add the same name, with the node id in front:

```
| eosc-example: EOSC Node | Example
```

If the Tripartite list does not include the node yet, do not make a name up. Leave both name files alone. `test_nodes.py::test_every_node_has_a_scoped_approved_name` then fails and names the node, which is intended: the node is not ready to be published. In the meantime, check it in a scratch folder with `--url https://eosc-node.example.org/` (see "Ad hoc URLs" in section 5), which needs no configuration.

Step 2: check the configuration, offline.

```
| uv run pytest -q
```

A missing scoped name, an id that is not a valid filename, a URL that is not `https`, an `eosc_page` outside `eosc.eu`, or a landing page already used by another node each fail with a message naming the problem.

Step 3: trial run for the new node only. This contacts one site.

```
| uv run basic-check collect --only eosc-example --results /tmp/trial
| uv run basic-check assess --only eosc-example --results /tmp/trial
| uv run basic-check show eosc-example --results /tmp/trial
```

Open `/tmp/trial/index.html` or `/tmp/trial/results.md` and read the new row. If the page returned `ERROR` (robots.txt, a 403, a DNS failure), stop there. An `ERROR` row is not a result worth publishing.

Step 4: put the node into the published run. There are two ways.

A — add the one node to the current run (one site contacted, and already done). Reuse the evidence from step 3 rather than fetching the page again:

```
cp /tmp/trial/evidence/eosc-example.json results/evidence/
cp /tmp/trial/evidence/screenshots/eosc-example.png results/evidence/screenshots/
uv run basic-check assess --only eosc-example --skip Italy \
--run live-2026-10-01-plus-eosc-example
```

The report covers every node. A **Mixed freshness** banner says that only `eosc-example` is new and that the other rows are reused from the earlier capture, with its date. `--skip Italy` keeps the same scope as the published run; drop it once Italy's page can be collected.

B — a complete new run (every node contacted once). Use this when the existing evidence is old enough that a single timestamp is worth more than leaving the other sites alone:

```
uv run basic-check run --skip Italy --results /tmp/new-run --run live-2026-10-01
# review /tmp/new-run/results.md and index.html, then:
rm -rf results/evidence
cp -r /tmp/new-run/evidence results/evidence
uv run basic-check assess --skip Italy --run live-2026-10-01
```

The last command re-renders the reports in `results/` from the evidence you reviewed. It makes no requests.

Either way, `assess` must end with **exit code 0**. Exit 2 means a configured node has no evidence, so the table is incomplete.

If `assess` warns "**Evidence from a different URL**". It means a node's URL in `nodes.yaml` is not the page its evidence was collected from. That is the case for BBMRI-ERIC as of 25 September 2026: `nodes.yaml` has the `dev3`. address, but the published evidence is from the old one. The warning also appears as a banner in both reports. Either collect that node again first (example 2), or assess with the node list the evidence was collected with, plus the new entry:

```
git show 53081f6:nodes.yaml > /tmp/nodes-published.yaml
# append the eosc-example entry from step 1 to /tmp/nodes-published.yaml
uv run basic-check assess --nodes /tmp/nodes-published.yaml --only eosc-example \
--skip Italy --run live-2026-10-01-plus-eosc-example
```

Step 5: re-run the tests against the new published run.

```
uv run pytest -q
```

Two tests read the committed evidence. Each exists to make a change visible, not to be satisfied mechanically:

- `test_privacy.py::test_the_committed_evidence_publishes_no_personal_address_or_phone` must pass as it stands. If it fails, personal data is about to be published. Stop and find out why; never change the test to let it through.
- `test_names.py::test_the_official_names_match_the_nodes_that_show_them` pins which nodes display their approved name (five, for the 24 September run). If the new page shows its name, add its id to the expected set and update the docstring to say why.

Step 6: update what quotes the published figures, then commit. `results/` is regenerated, but these are written by hand:

- a review document for the new run, like [results/REVIEW-2026-09-24.md](#);
- the headline tally in [README.md](#) ("120 cells: 44 PASS · 6 FAIL · 70 review");
- section 10 of this document ("The published figures"), and the node list in section 5 ("The node list — a YAML file").

```
git status
git add nodes.yaml checklist/approved-names.txt checklist/approved-names-scoped.txt \
    results README.md docs tests
git commit -m "Add eoscc-example; publish run live-2026-10-01-plus-eoscc-example"
git push
```

Example 2 — changing a node's landing page URL

The example is the change actually made on 25 September 2026: BBMRI-ERIC moved from <https://www.bbmri-eric.eu/eoscc-node-bbmri-eric/> to <https://dev3.bbmri-eric.eu/eoscc-node-bbmri-eric/>.

Which files change. One line in one file: `url` for that node in [nodes.yaml](#). Nothing else in the repository stores a node's URL. Add a comment recording the old address and the date, so the published evidence can still be traced:

```
- id: bbmri-eric
  name: BBMRI-ERIC
  # URL changed on 25 September 2026, from
  # https://www.bbmri-eric.eu/eoscc-node-bbmri-eric/
  url: https://dev3.bbmri-eric.eu/eoscc-node-bbmri-eric/
  eoscc_page: https://eoscc.eu/building-the-eoscc-federation/eoscc-node-bbmri-eric/
```

Change `eoscc_page` as well only if the node's own entry on [eoscc.eu](#) moved; a new landing page does not usually mean a new [eoscc.eu](#) entry. The name files change only if the node's approved name changed. Keep the `id`: it names the evidence file, and changing it would make the node look new.

Step 1: check the configuration.

```
uv run pytest -q
```

Step 2: trial run for that node only. This contacts one site.

```
uv run basic-check collect --only bbmri-eric --results /tmp/trial
uv run basic-check assess --only bbmri-eric --results /tmp/trial
uv run basic-check show bbmri-eric --results /tmp/trial
```

Check that the page actually loaded before going further. On 25 September the `dev3.` host's `robots.txt` disallowed every path, the tool honoured it, and all ten points were `ERROR`. A result like that is not worth publishing: ask the node for its public address, or for the checker to be allowed, and stop there.

Step 3: replace that node's evidence in the published run. Reuse the trial evidence rather than fetching the page again:

```
cp /tmp/trial/evidence/bbmri-eric.json results/evidence/
cp /tmp/trial/evidence/screenshots/bbmri-eric.png results/evidence/screenshots/
uv run basic-check assess --only bbmri-eric --skip Italy \
    --run live-2026-10-01-new-bbmri-url
```

If the new capture has no screenshot, because the page could not be rendered, delete `results/evidence/screenshots/bbmri-eric.png` rather than keep the old page's image. The report covers every

node, with a **Mixed freshness** banner naming `bbmri-eric` as the only row fetched again. For a completely fresh table instead, follow path B of example 1 step 4.

Why the order matters. `assess` labels each row with the URL in `nodes.yaml`, but takes the verdicts from the evidence on disk. Between step 1 and step 3, the two describe different pages. `assess` now detects this. It prints a warning naming the node and both URLs, puts an **Evidence from a different URL** banner in `results.md` and `index.html`, and records the pair under `url_mismatch` in `results.json`. Step 3 clears it, because the evidence now comes from the configured URL.

To rebuild the old published run instead, without the warning, use the node list it was collected with. This reproduces the committed `results/` exactly, apart from the generation time:

```
git show 53081f6:nodes.yaml > /tmp/nodes-2026-09-24.yaml
uv run basic-check assess --run live-2026-09-24-no-italy --skip Italy \
    --nodes /tmp/nodes-2026-09-24.yaml
```

Step 4: tests, review, commit. As in example 1, steps 5 and 6: run `uv run pytest -q`, check the two tests that read the committed evidence, write the review, update the quoted figures, and commit `nodes.yaml`, `results/` and the documents together.

Example 3 — moving to a new revision of the checklist

There is no URL to change. The tool never downloads the checklist document. A copy of it is committed in `checklist/`, and three things connect the tool to it:

Where	What it says
<code>src/basic_check/cli.py</code> , the line <code>DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.0.yaml"</code>	Which checklist revision is used when <code>--checklist</code> is not given. This is the one line to change to make a new revision the default. The help texts, the tests and the reports all follow it.
<code>checklist/vX.Y.yaml</code> : <code>checklist_version</code> , <code>checklist_date</code> , <code>source_document</code> , <code>source_file</code> , <code>source_sha256</code>	Which document that revision was transcribed from, and its SHA-256 hash.
<code>--checklist / -c <file></code> on <code>assess</code> , <code>run</code> and <code>points</code>	A different revision for a single run, without changing the default.

When the originating document changes, you therefore add a new file for the new revision and point the default at it. `v3.0.yaml` is never edited: the published reports name the file they were produced from.

The example assumes a v3.1 dated 15 October 2026. Use the real version, date and filenames.

Step 1: commit the new document next to the old one.

```
cp ~/Downloads/20261015_Node_Landing_Page_Verification_Checklist_v3.1.pdf checklist/
sha256sum checklist/20261015_Node_Landing_Page_Verification_Checklist_v3.1.pdf
# on macOS: shasum -a 256 checklist/20261015_Node_Landing_Page_Verification_Checklist_v3.1.pdf
```

The committed v3.0 file is a PDF rendering of the circulated `.docx`. Commit the `.docx` itself if you prefer: `source_file` may name either.

Step 2: create the new revision's file from the old one.

```
cp checklist/v3.0.yaml checklist/v3.1.yaml
```

In `checklist/v3.1.yaml`, change the header:

```
checklist_version: "3.1"
checklist_date: "2026-10-15"
source_document: 20261015_Node_Landing_Page_Verification_Checklist_v3.1.docx
source_file: 20261015_Node_Landing_Page_Verification_Checklist_v3.1.pdf
source_sha256: <the hash printed in step 1>
```

The version must match the filename (`v3.1.yaml` declares "3.1"); a test enforces this.

Step 3: compare every point with the new document. No tool can do this step for you. For each point in `v3.1.yaml`:

- If the wording changed, update `title` and `requirement`, which are quoted word for word. Then read the `check_*` function named in `implemented_by`, in `src/basic_check/checks.py`, and change it if it no longer applies the new wording.
- If what a script can decide changed, update `decidable` and `decidable_note`.
- If a point was **added**, write a new `check_<id>` function in `checks.py`, call it from `run_all()` in `checklist` order, name it in the point's `implemented_by`, and add tests for it to `tests/test_checks.py`.
- If a point was **removed**, delete its entry, its function and its tests.

Step 4: try the new revision offline, before switching. `-c` applies it to one run only. Re-scoring saved evidence contacts no one:

```
uv run basic-check points -c checklist/v3.1.yaml
mkdir -p /tmp/v31 && cp -r results/evidence /tmp/v31/
git show 53081f6:nodes.yaml > /tmp/nodes-2026-09-24.yaml
uv run basic-check assess -c checklist/v3.1.yaml --results /tmp/v31 \
--nodes /tmp/nodes-2026-09-24.yaml --skip Italy --run trial-v3.1
```

Compare `/tmp/v31/results.md` with `results/results.md`. Any difference comes from the new rules alone, since the evidence is the same.

Step 5: make it the default, and run the tests. In `src/basic_check/cli.py`:

```
DEFAULT_CHECKLIST = ROOT / "checklist" / "v3.1.yaml"
uv run pytest -q
uv run basic-check --help # now says "against checklist v3.1"
```

The tests check the new file's hash, that its version matches its filename, and that every point maps to exactly one `check_*` function and every function to one point. They also check that `v3.0.yaml` and its document are still intact, so older runs can still be rebuilt with `-c checklist/v3.0.yaml`. What they cannot check is whether a rule is right: that is step 3.

Step 6: produce and publish the results. The default revision is now `v3.1`, so no `-c` is needed. Choose between the same two paths as in example 1:

```
# fresh evidence from every node:
uv run basic-check run --skip Italy --results /tmp/new-run --run live-2026-10-20-v3.1
# review, then:
rm -rf results/evidence && cp -r /tmp/new-run/evidence results/evidence
uv run basic-check assess --skip Italy --run live-2026-10-20-v3.1
```

Alternatively, re-score the published evidence against v3.1 without contacting anyone, as in step 4 but writing to [results/](#). The report then says v3.1, while the capture date is still that of the evidence.

The reports now link to [results/checklist-v3.1.html](#). The old [results/checklist-v3.0.html](#) stays behind; remove it, and update the places that name v3.0 in prose or links:

```
git rm results/checklist-v3.0.html
rg -n "v3\0" README.md docs checklist/README.md .github
```

In [README.md](#), the table of checklist points links each point to [results/checklist-v3.0.html#p....](#) Those links break once the file is removed, so change them to v3.1.

Step 7: tests, review, commit. As in example 1, steps 5 and 6. If any `check_*` function changed in step 3, also update its decision procedure in section 5 of [ANALYSIS-WORKFLOW.md](#), which describes each point's rule branch by branch. Commit the new document, [checklist/v3.1.yaml](#), [cli.py](#), any changed checks and tests, [results/](#) and the documents together, so the commit shows the whole change.

Sources

- Checklist: *Node Landing Page Verification Checklist v3.0*, 15 September 2026, committed as [checklist/20260910_Node_Landing_Page_Verification_Checklist_v3.0.pdf](#) and transcribed to [checklist/v3.0.yaml](#) with its SHA-256 pinned
- EOSC Federation node index: <https://eosc.eu/building-the-eosc-federation/>
- Repository: <https://github.com/marioarele/eosc-basic-compliance>
- Figures in section 10, current run: [results/results.json](#), run [live-2026-09-24-no-italy](#), collected 24 September 2026, reviewed in [results/REVIEW-2026-09-24.md](#)
- Figures in section 10, previous run: [results/results.json](#) at commit [47f08af](#) (unchanged since [014682c](#)) — evidence collected 21 September 2026 13:04–13:07 UTC, report regenerated 17:38 UTC
- Approved node names: [checklist/approved-names.txt](#) (thirteen names, SHA-256 [871161a5...](#)) and the node-scoped variant [checklist/approved-names-scoped.txt](#)
- Test counts, line counts, runtimes, name-list results and command options in this document were measured on 25 September 2026 against a clean clone of the repository state it is committed with (the test count re-checked on 26 September at [0cc33c2](#)), not carried over from the previous edition